



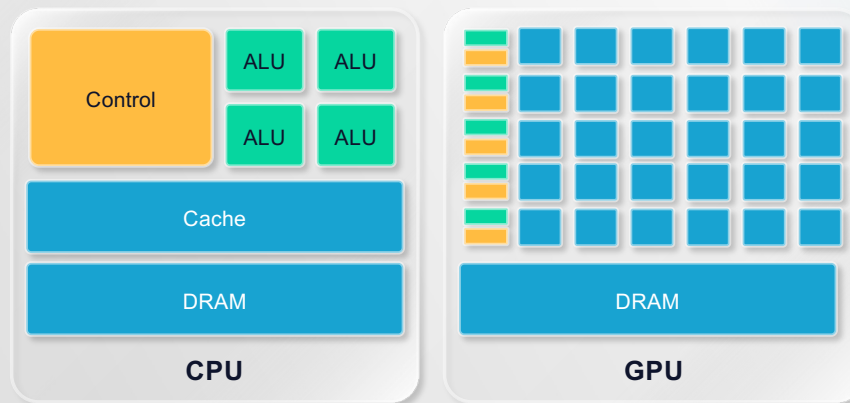
Enterprise AI Fundamentals

The Basics

Changing Requirements: CPUs to GPUs

Parallel Compute Needs Parallel Storage

CPUs vs GPUs (How They Work)



Few cores, flexible,
sequential processing

Lots of cores, simple math,
parallel processing

What is CUDA?

- CUDA = NVIDIA's programming model for GPUs
- Allows developers to run workloads **in parallel across thousands of cores**

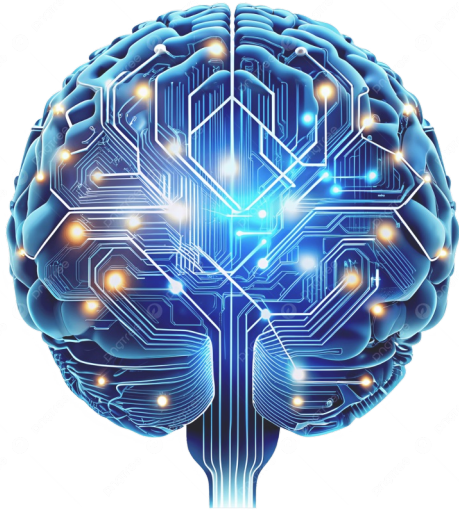
AI workloads = millions of similar math operations → GPUs win

Parallel Compute Changes Storage Access

Parallelism: ability to access and process many pieces of data at the same time instead of one at a time.

Category	CPU Access Pattern	GPU Access Pattern
Data Access Style	Sequential or lightly parallel	Massive parallel access
I/O Pattern	Small, random reads/writes	Large, concurrent, high-throughput reads
Concurrency	Low (tens of threads)	Extremely high (thousands of threads)
Latency Sensitivity	High (per request)	Lower per request, but needs sustained throughput
Throughput Requirement	Moderate	Extremely high (GB/s → TB/s)
Storage Design Fit	Latency-optimized systems	Throughput + concurrency-optimized systems
Bottleneck Risk	CPU waits on latency	GPU starvation

Models: The Brains of AI



VAST

WHAT THEY DO

1. Understand
2. Reason
3. Generate

TYPE OF MODELS

Language Models

Audio Models

Vision Models

Multimodal Models

HOW THEY'RE MADE

Step 1:

Feed it data

Step 2:

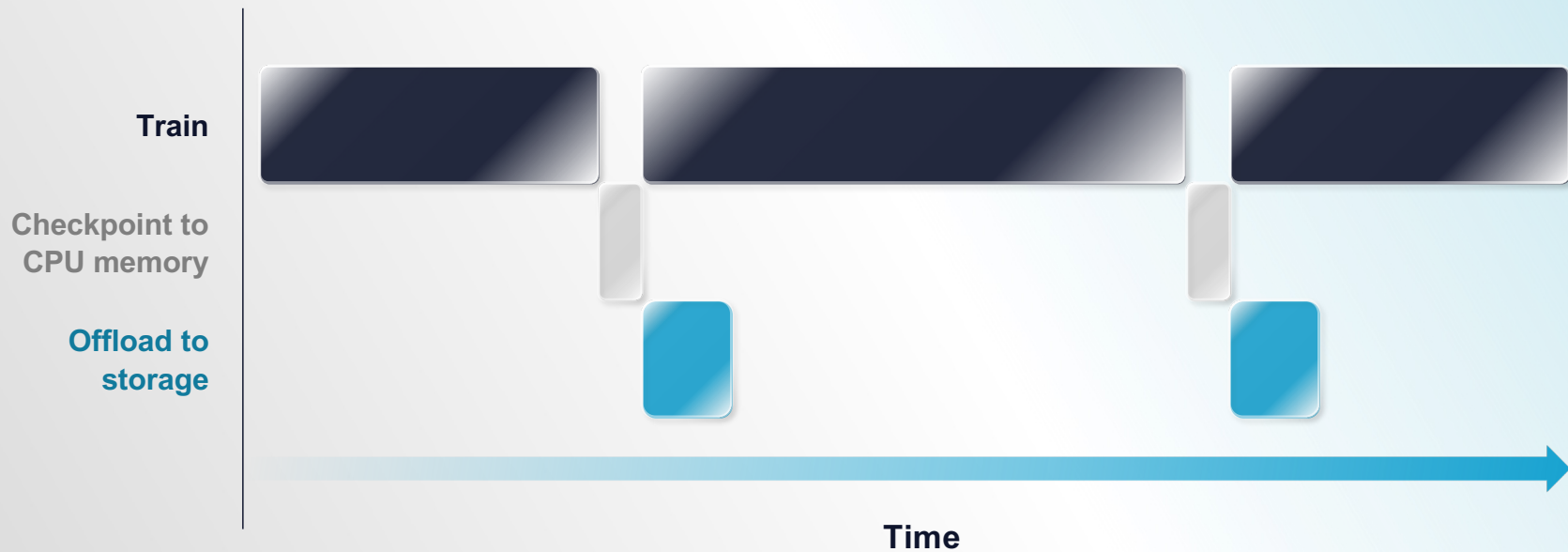
Train it to predict

Step 3:

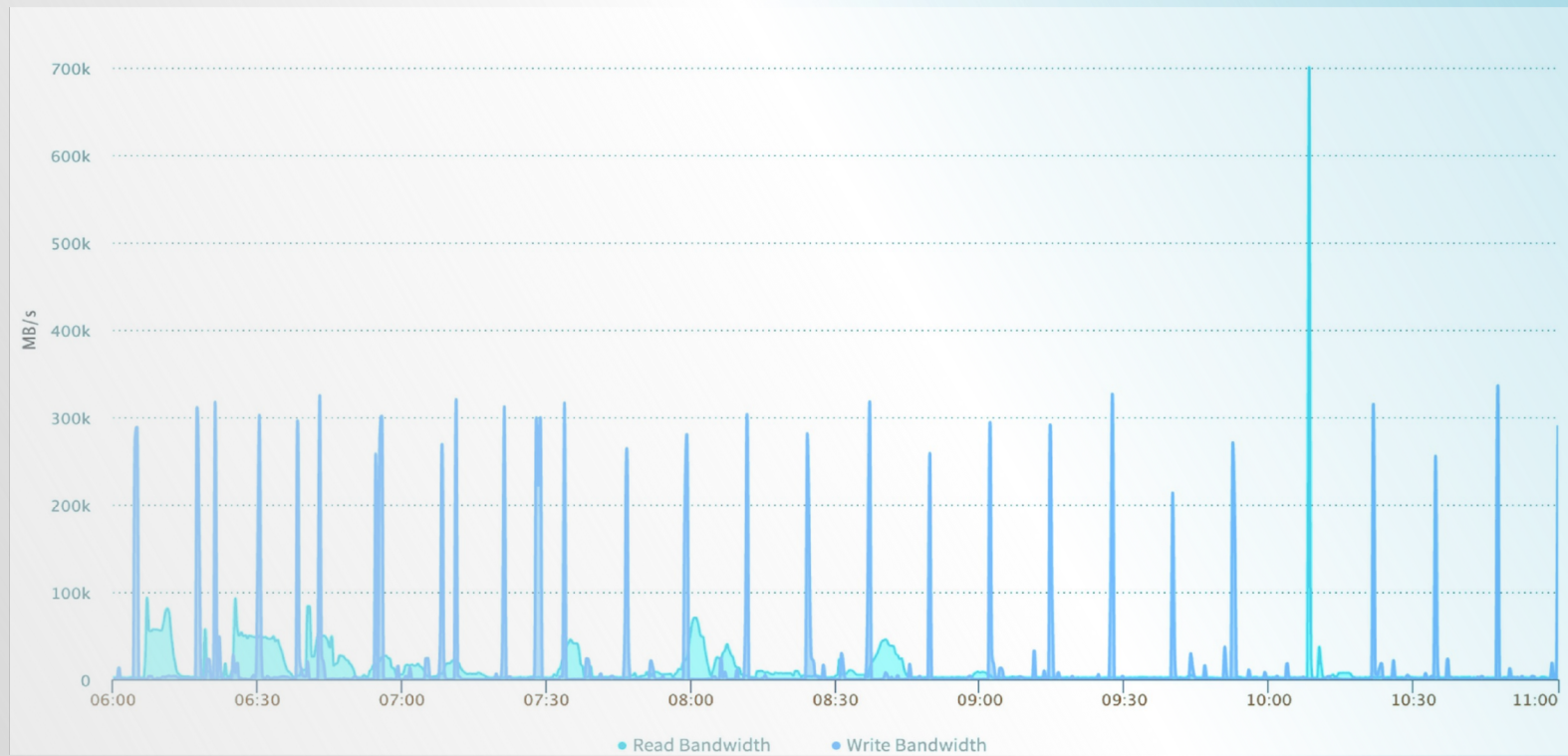
Fine-tune it

Checkpoints Keep Training on Track

Asynchronous Checkpoints



Large Model Training



Training and Inference



Training

- Build the model
- Studying for the test
- Learns patterns from tokens



Inference

- Use the model
- Taking the test
- Generates new tokens

Token: the smallest unit of input a model processes, representing a piece of data—like a group of words or frame of a video—converted into a format the model can compute on.

Fine Tuning

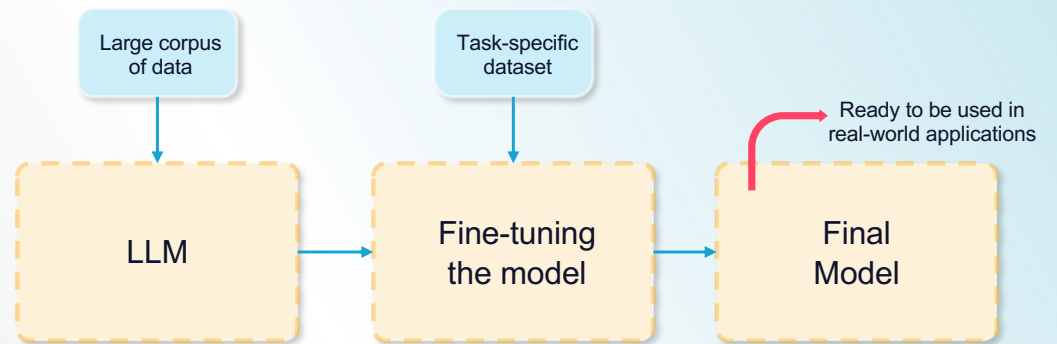
Optimize Models for your Business

Goal:

Improve how a model responds

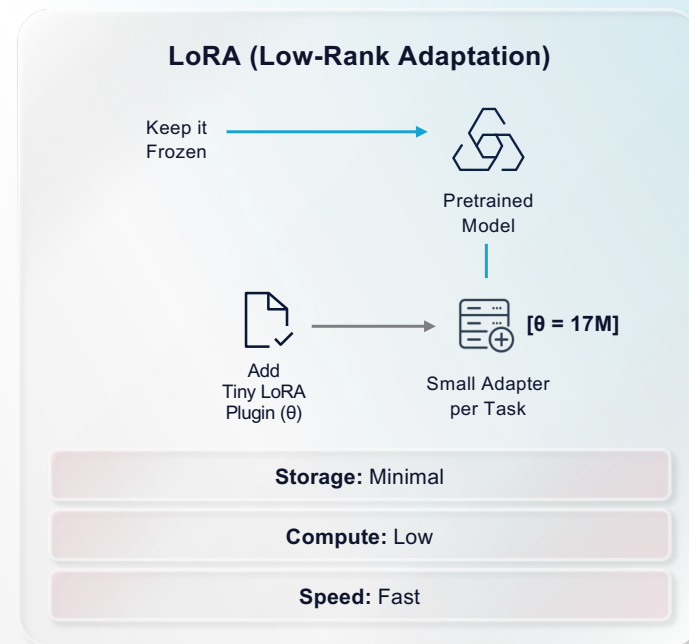
How:

- Start with a pre-trained model
- Create examples of what good looks like
- Train



Fine Tuning – How it's Evolved

Customize More Efficiently



Fine Tuning vs Training

Different Goals, Different Workloads

	Training	Fine-Tuning
Purpose	Create a general model	Specialize for a task
Compute Load	Extremely high	Moderate to low
Time Required	Weeks–months	Hours–days
Data Size	Massive (TBs–PBs)	Small (MBs–GBs)
Infrastructure	Thousands of GPUs	Single to dozens of GPUs
Frequency	Rare	Frequent / iterative

Agents: What are they?

Components of an Agent

LLM

How the agent interprets inputs, generates responses, and reasons.

Profile

Role, goals, and how it should approach its tasks. Delivered in prompt or in code.

Memory

Short-term memory (recent context and input) and long-term memory (prior actions or gained knowledge).

Tools

APIs to connect agent to users, agents, databases, sensors, and other systems. How they observe and act.

Feedback

Improve through feedback loops—capturing outcomes and using them to get better over time.

OpenClaw Example

Make Your Own Personal Assistant Agent

LLM

Connect to
Claude Opus 4.6
LLM for
reasoning.

Profile

SOUL.md:
Who your agent is.
AGENTS.md:
How it operates.
USER.md:
Who you are.

Memory

MEMORY.md:
Long-term
memory. Things
it's learned about
you over time.

Tools

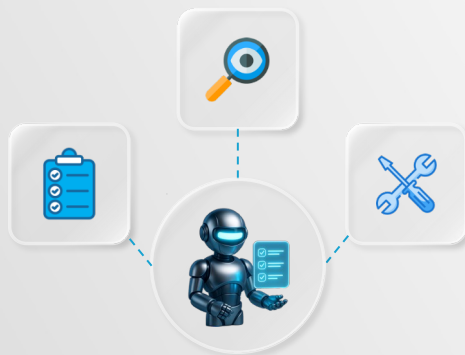
SKILLS.md:
what your agent can
do. Send WhatsApp
messages, send
emails, update your
calendar.

Feedback

"Too formal." "I
wouldn't say it
that way." "More
concise."

Agents: What do they do?

Observe, Plan, Act



Observe

Gather information from their environment:

- User interactions
- Systems (data bases, RAG, sensors)
- Memory – context from experience

Plan

Evaluate problems and prioritize actions

- Goals from users
- Role defined in the agent's profile
- Reasoning through LLM
- Context from past interactions

Act

Execute planned actions

- Tools to interact with digital systems and data
- Delegation to other agents
- Get clarification from users when needed

OpenClaw Example

Observe, Plan, Act



Observe

- New email from prospect asking for follow-up
- Reviews prior thread + meeting notes
- Identifies key interest: AI / data platform

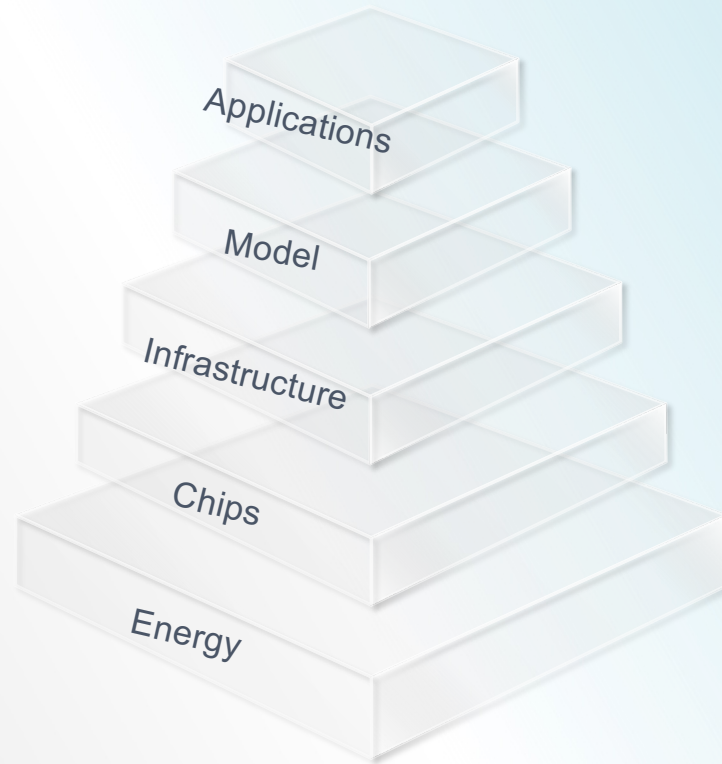
Plan

- Selects relevant follow-up message
- Personalizes based on discussion
- Decides next step: propose workshop

Act

- Drafts and sends reply
- Includes tailored content
- Suggests next meeting

AI is a 5 Layer Cake

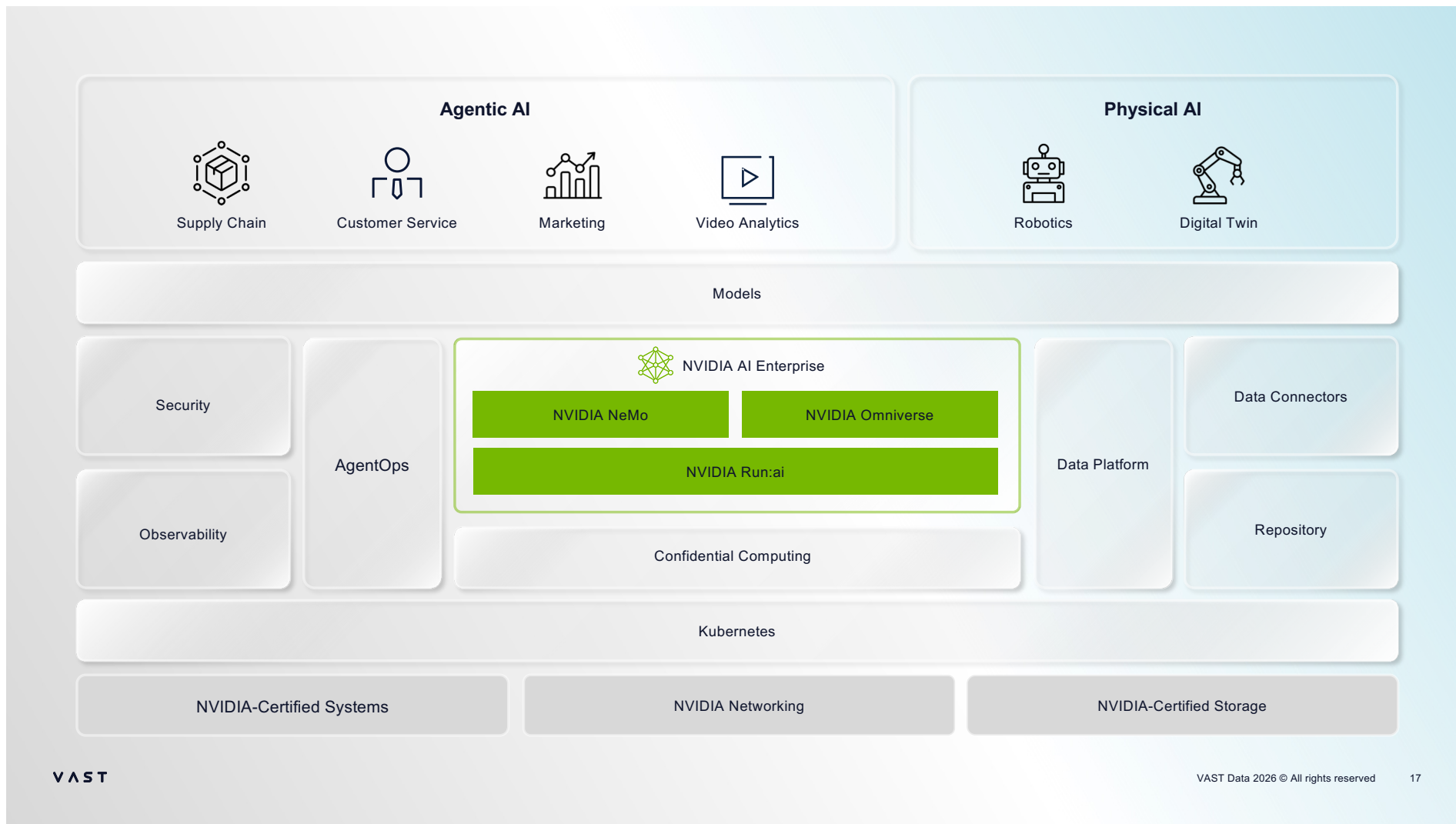




What is an AI Factory?

An AI factory is a specialized computing infrastructure designed to create value from data by managing the entire AI life cycle, from data ingestion to training, fine-tuning, and high-volume AI inference. The primary product is intelligence, measured by token throughput, which drives decisions, automation, and new AI solutions.

[Nvidia's Glossary](#)





What is an AI Data Platform?

The NVIDIA AI Data Platform is a customizable reference design for a new class of AI infrastructure. It transforms enterprise storage systems by integrating NVIDIA-accelerated computing, networking, and AI software to power generative AI systems with near real-time business insights via Retrieval Augmented Generation (RAG)

[Nvidia's Definition](#)

AI Factory vs AI Data Platform

Different Scopes

Feature	Nvidia AI Data Platform	Nvidia AI Factory
Goal	Intelligent knowledge retrieval & semantic understanding of enterprise data for RAG.	Building & deploying a full stack solution for the entire AI lifecycle.
Main Focus	Continuous data ingestion, embedding, & indexing for AI consumption.	Producing intelligence at scale; unifying AI lifecycle stages (training, fine-tuning, inference).
Typical Workloads	Connecting AI Agents to enterprise data, semantic search of multimodal data for Gen AI.	Agentic AI, Physical AI, HPC, scientific simulations, autonomous decision-making, real-time data analysis.
Core Function	Transforms enterprise storage into an AI-ready knowledge retrieval system.	Provides a comprehensive, purpose-built system for intensive AI compute needs.
AI Lifecycle Stages	Focused on data preparation, retrieval, and augmentation for inference.	The entire AI lifecycle: data ingestion, pre-training, fine-tuning, and long-thinking inference.
Relationship	Can be a foundational design guide or a component inside an AI Factory.	Can implement capabilities of an AI Data Platform for knowledge retrieval.

What is a pipeline?

Triggers:

Starts a pipeline or advances a pipeline to the next

Examples:

- Data is created
- Data is modified
- Scheduled time
- Process completed
- Event/API

Function:

Discrete processing step in response to a trigger

Examples:

- Extract (pull data from a source)
- Transform (normalize data formats)
- Load (inserts into data warehouse)
- Chunk (breaking data into pieces)
- Embed (convert data to vector)

Trigger



Starts Pipeline

Function



Trigger



Output/Input

Function



Trigger



Output/Input

Function



Final Output

What is a RAG?

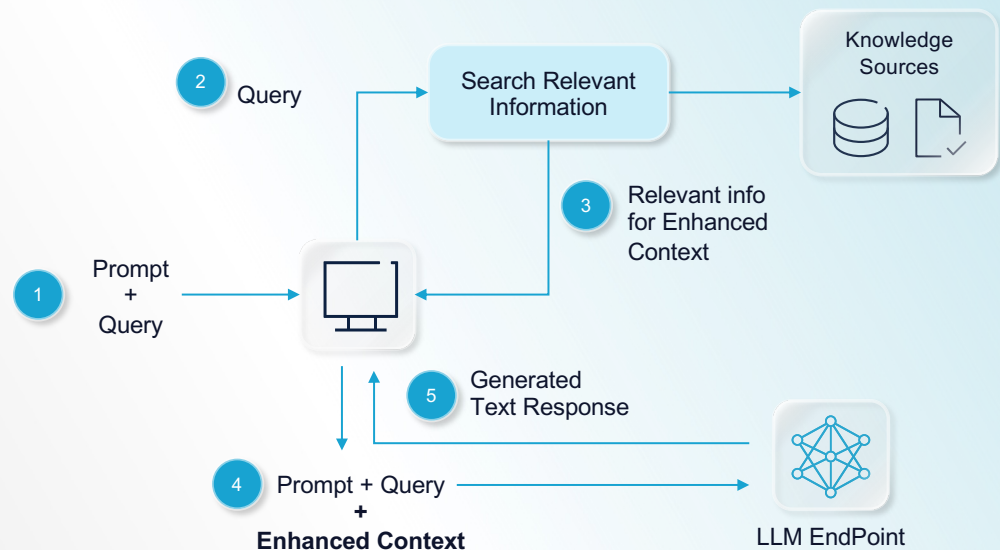
RAG (Retrieval-Augmented Generation) = giving an AI model access to your data when it needs it

Without RAG:

- Model is generic
- Can be outdated
- May hallucinate

With RAG:

- Answers are grounded in your data
- Up-to-date
- More accurate and trustworthy



How does embedding work?

Embedding = converting data into a vector so its meaning can be compared and processed mathematically

Extract Pull raw data from source systems

Prepare Extract useful information and standardize it across different sources.

Chunk Break the data into smaller, meaningful pieces that are easier to process and retrieve.

Enrich Add context to each piece of data using metadata like source, time, or labels.

Load Organize vectors, adding metadata, and building indexes to enable fast similarity search.

How does retrieval work?

Retrieval = finding the most relevant pieces of data to answer a question

It's not keyword search—it's **semantic matching** (meaning, not exact words).

Embed

Convert the query/question into a vector

Search

Look for the closest match to the query vector among the stored vectors

Return

Select the most relevant chunks (pieces of documents, not entire documents)

Re-Rank

Optional second pass where a second model orders retrieved results to improve accuracy

Assemble

Package the selected chunks into a prompt to deliver the final answer

What is a vector?

A vector = a list of numbers

Vectors represent meaning by encoding relationships between words, concepts, and context as numbers. It's not keyword search, it's **semantic matching** (meaning, not exact words).

"dog" → [0.12, -0.87, 0.44, ..., 0.03]

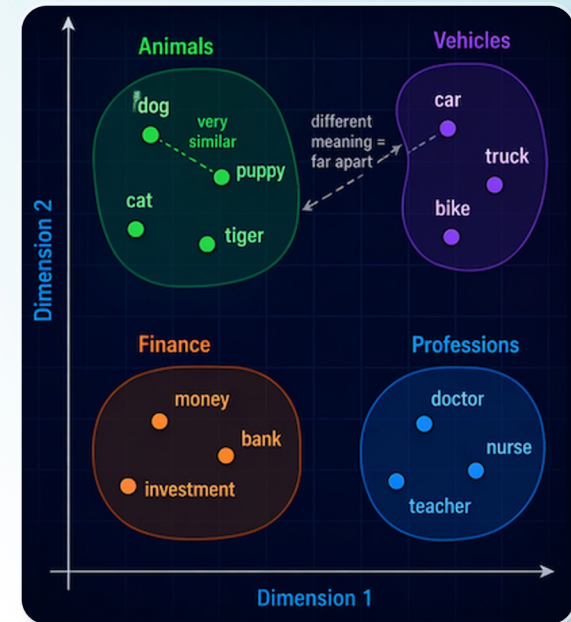
How many numbers? Determined by the number of dimensions, changes by model:

Dimensions	Speed	Accuracy
Low (256–512)	Very fast	Lower
Mid (768–1536)	Balanced	Strong
High (2K+)	Slower	Best

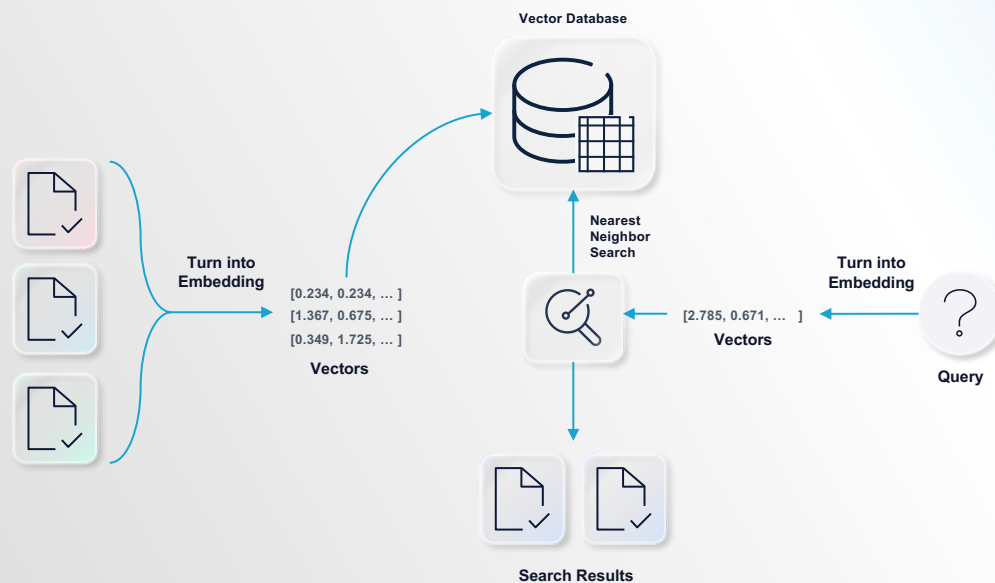
Similar meaning = similar vector (i.e. dog and puppy)

How similar or different = distance

Each vector represent a chunk of data (a group words, not a single word). Chunk sizes change based on model



How does Similarity Search work?



Distance functions measure how “close” two vectors are

Closer = more similar meaning

Farther = less similar

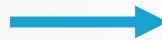
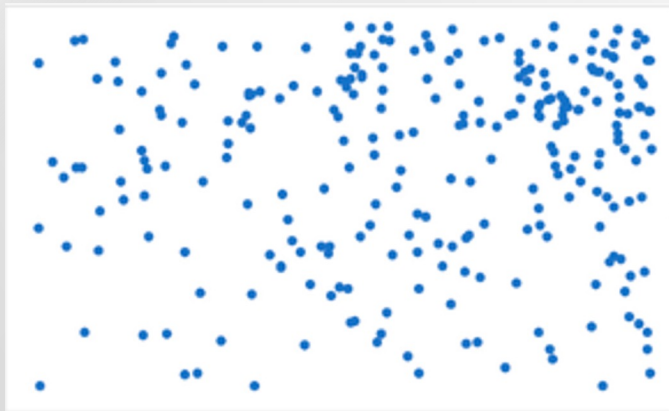
This is how AI decides:

“Which data is most relevant?”

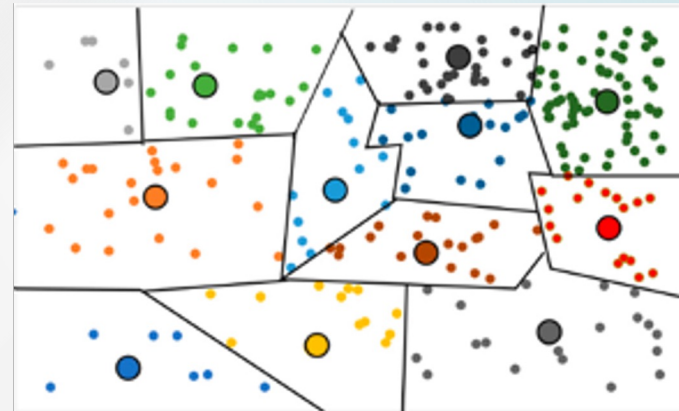
How does AI find the right vectors?

ANN (Approximate Nearest Neighbor) accelerates vector search by pointing the the group of vectors where you're most likely to find the best answer. The tradeoffs are a reduction in accuracy and tuning required to balance performance, accuracy, and resources.

Brute Force = search every vector
High accuracy, slower



ANN = search the most likely group
Faster, can miss outliers



Common Types:

HNSW (graph) → accuracy

IVF (clusters) → scale

PQ (compression) → memory efficiency

*Hint – DASE enables something different.

What is context?

Context is the information an AI model uses to understand and respond correctly

Training = what the model learned before

Context = what the model sees right now

Context = Prompt + History + Retrieved Data + Instructions

Prefill Worker

KV CACHE

Read and understand all context

Decode Worker

KV CACHE

Generate output using that context

What is KV Cache?

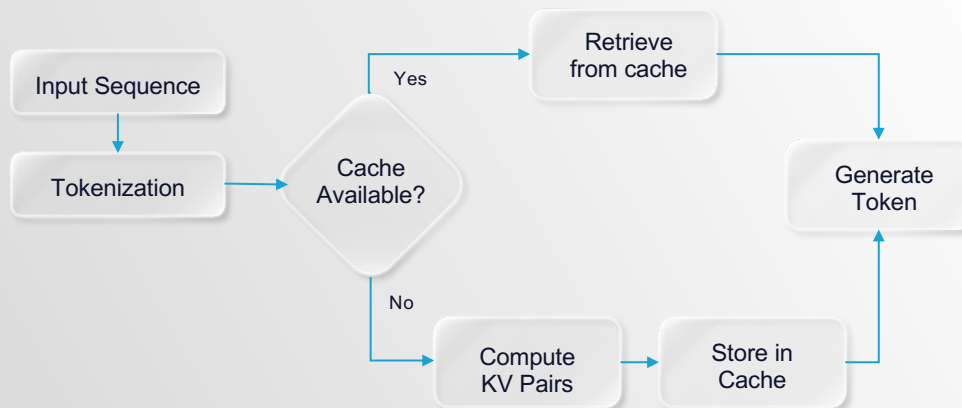
KV cache is the model's working memory structure

KV Cache stores context, so the model doesn't have to recompute it

- "K" = Keys – what is this token about?
- "V" = Values – what information is in there?

Stored per token during inference.

Architecture Diagram of KV caching



Where it matters most:

- Multi-turn conversations
- Long documents
- Agent workflows

Benefits:

- faster response for repeated context
- less GPU cycles for recalculating
- more GPU cycles for new requests

Challenges:

- Moves problem from compute to memory
- Not shared across nodes

Solution: Nvidia Dynamo

- Distributed KV Cache – share context across nodes
- Tiered memory – (HBM, Dram, NVMe)

FASTEST ACCESS

Optimized

INCREASING LATENCY

Access overhead grows

HIGHEST LATENCY

Inference bottleneck

G1 – GPU HBM

~Nanoseconds | Active KV

G2 – System DRAM

~10-100 Nanoseconds | Staging / KV

G3 – Local SSD / Rack-local

~Microseconds | Warm KV reuse

G4 – Shared Object / File

~Milliseconds | Cold or shared KV context

PEAK INFERENCE EFFICIENCY

Best perf/watt, perf/\$

RISING ENERGY & COST

Per token overhead increases

LOWEST EFFICIENCY

Limits AI inference scaling

VAST



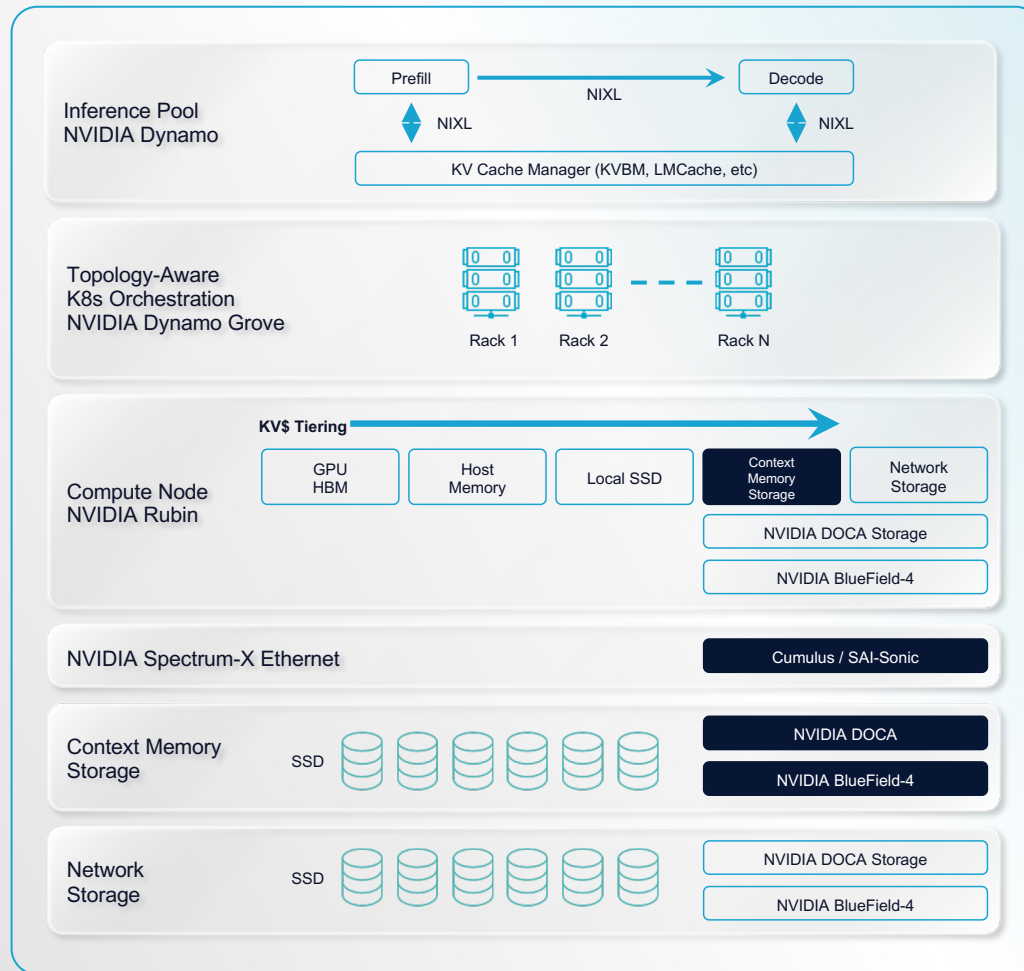
**NVIDIA Context
Memory Storage Platform**

Introducing NVIDIA Context Memory Storage (CMX)

NVIDIA BlueField-4-powered storage infrastructure
for the next frontier of AI

- AI-native, pod-level context storage tier
- Shared across GPUs for efficient context reuse
- Complements GPU memory, local, and networked storage
- Improves performance and scale, reducing TCO:
 - 5x higher throughput (tokens-per-second) than baseline without additional capacity
- Accelerated by BlueField-4, Spectrum-X Ethernet, DOCA, NIXL, and Dynamo
- Designated new class of storage by Nvidia

CMX



AI Data Sources

The Fuel

Data Source	Description	Access	How AI Interacts
Cameras, microscopes, satellites, audio sensors, robots, telescopes	Any client creating new unstructured data	NFS/S3 Clients	Embeds for search and retrieval
Existing documents, images, and videos	Existing corpus of unstructured data on legacy systems	Extract from external storage systems	Connects via APIs to ingest and embed
OneDrive, Salesforce, Confluence, Google Drive	Business apps with user-generated data	APIs to SaaS applications	Pulls documents, records, and knowledge for RAG – semi-structured
Inventory, orders, customer data, security logs, IoT sensors	Structured, real-time business data	SQL query to data warehouse	Queries data for context, heavy agent usage
User clicks, views, actions, tickets, orders, shipments, event triggers	Continuous real-time data feeds	Subscribe to streaming topic	Can be a combination of structured and unstructured, AI observability

Key Players in Enterprise AI

The Pitch:
Full stack solutions

The Reality:
Expensive,
fragmented stacks

Clouds



The Pitch:
End-to-end solutions

The Reality:
Legacy architectures
and missing pieces

Infrastructure Platforms



The Pitch:
AI where your
data already is

The Reality:
Stacks built for
batch analytics

Data Platforms



The Pitch:
Purpose built for search

The Reality:
Not built for scale
or real-time

Vector Databases



The Pitch:
AI needs maximum
storage performance

The Reality:
Create more problems
than they solve

HPC Storage



The Pitch:
Mature solutions
upgraded for AI

The Reality:
Viable as a point
solution at best

Enterprise Storage



AI Factory Pain Points



Scale limitations of legacy platforms create fragmentation and limits real-time response

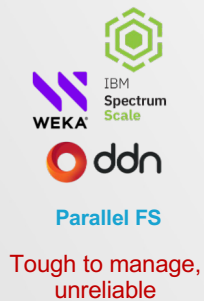
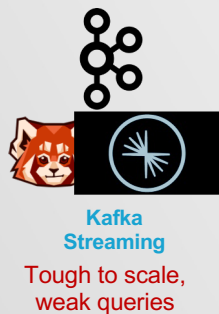
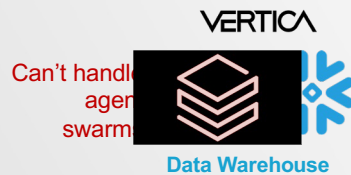


Fragmentation creates complexity



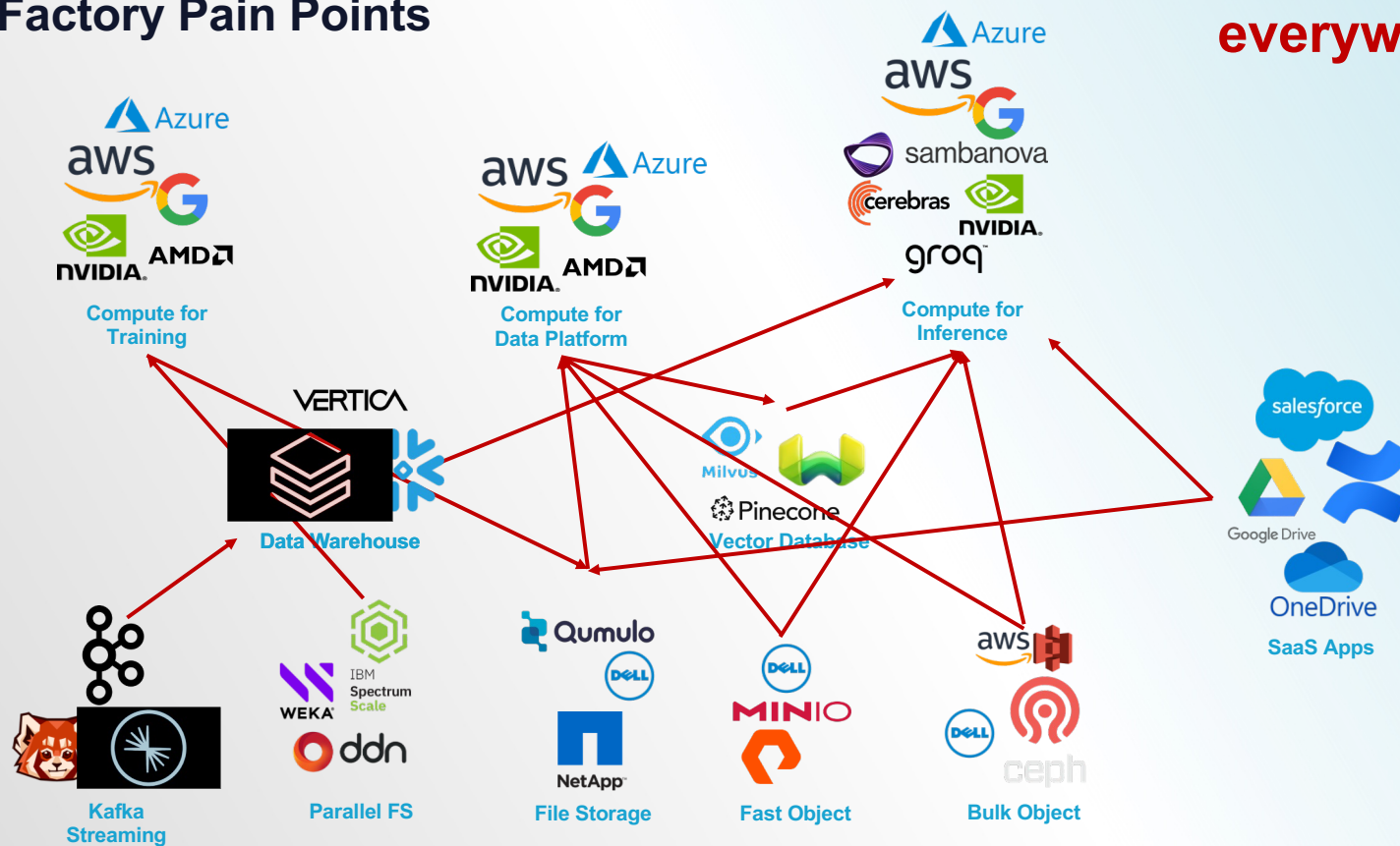
Complexity creates security vulnerabilities, loss of control, and loss of visibility

AI Factory Pain Points

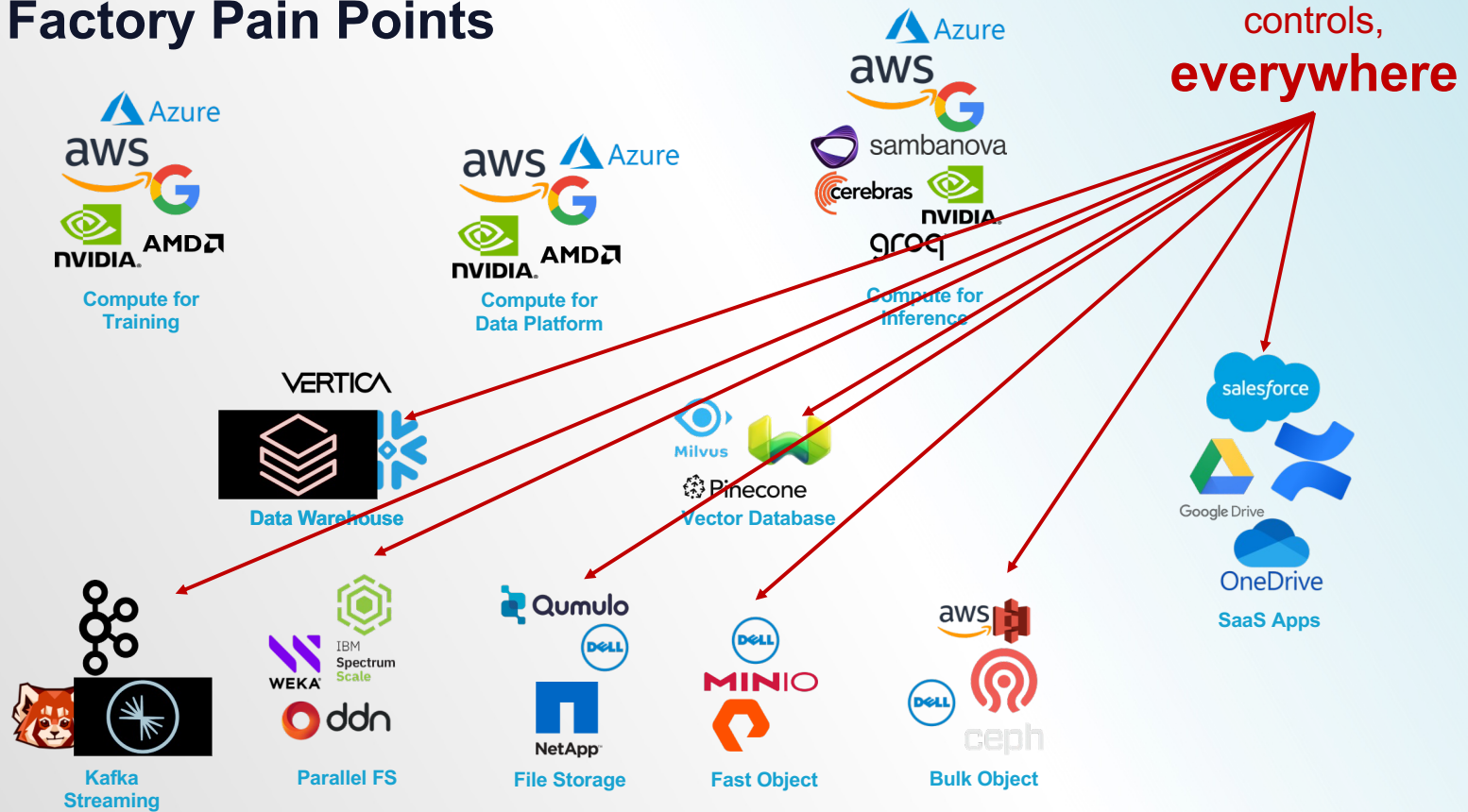


AI Factory Pain Points

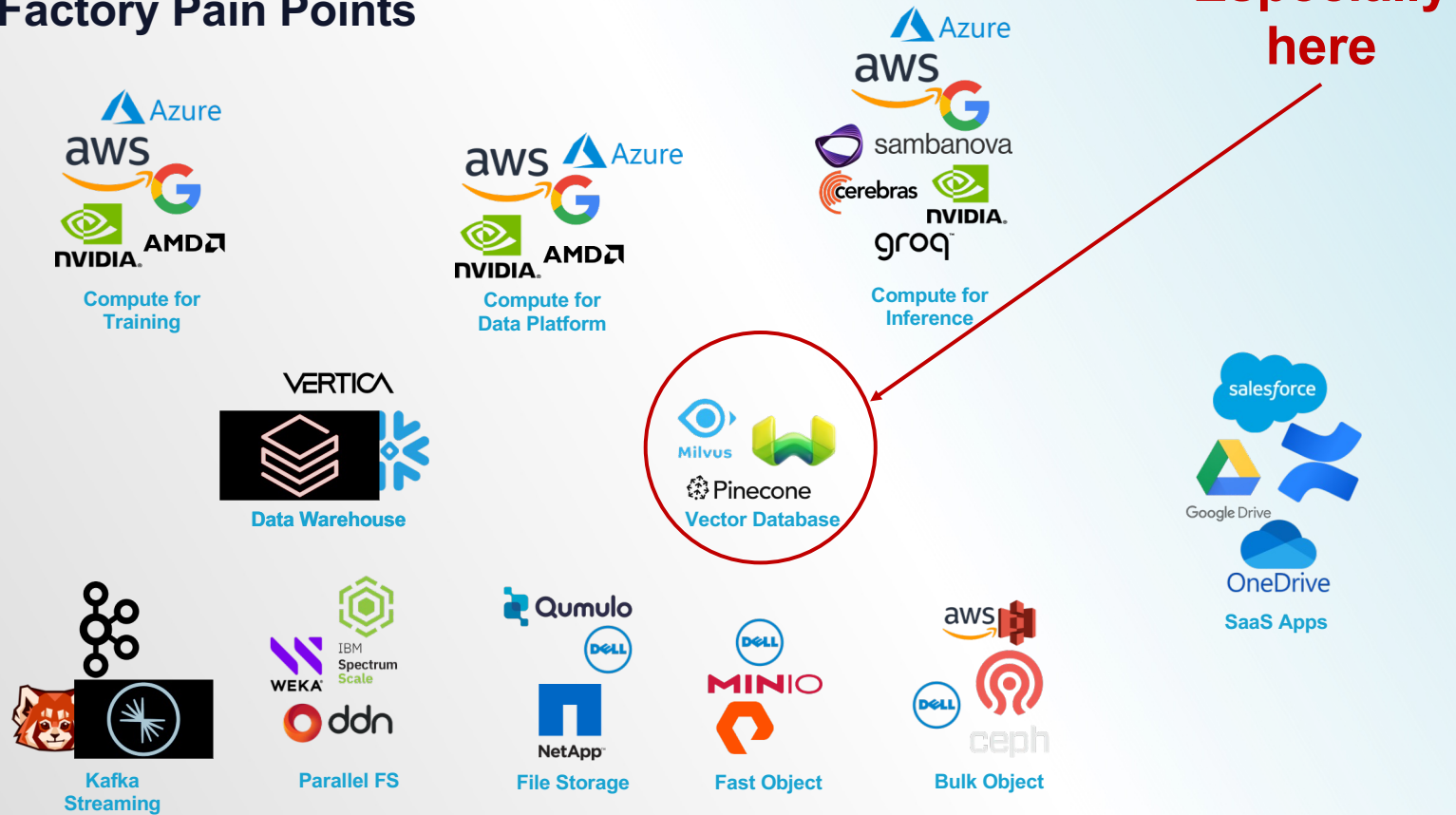
Data moves, **everywhere**



AI Factory Pain Points

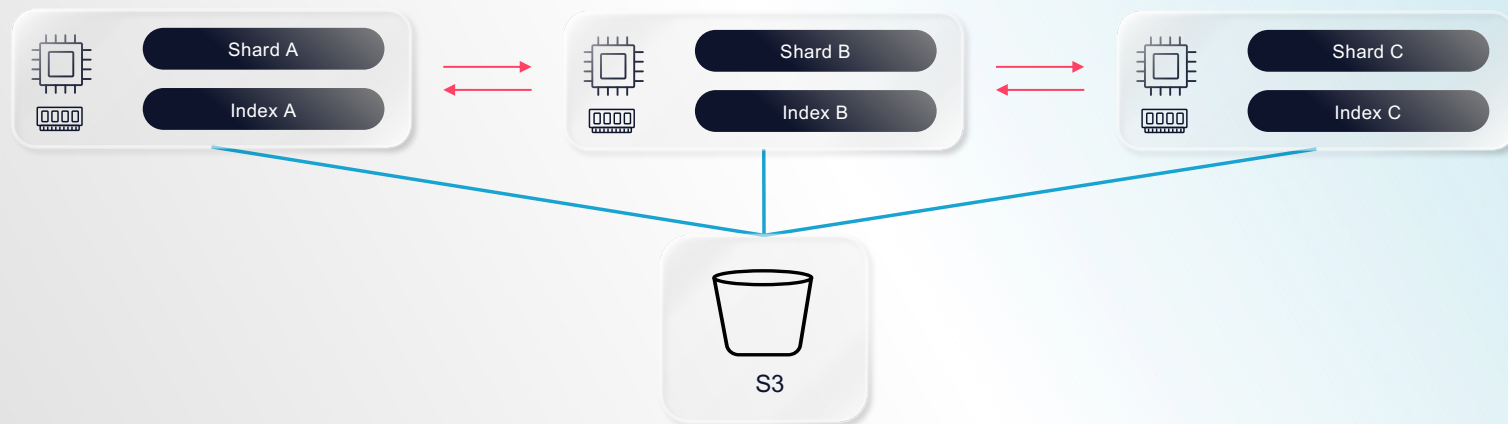


AI Factory Pain Points



The Vector Challenge

The Bottleneck of Inference



Scaling = Sharding

East-west traffic limits vector count.

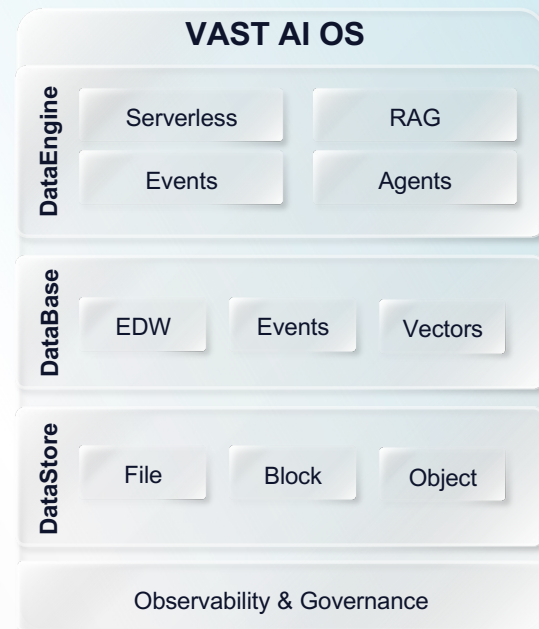
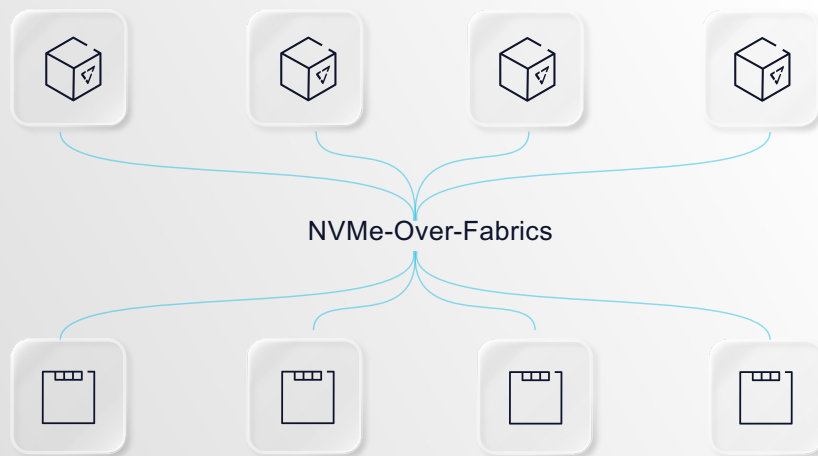
Memory-bound

Latency explodes on disk.
Expensive to scale.

Updates are Slow

Nodes must coordinate updates.
Not real time.

VAST Value: DASE + AI OS



Scale

Real-Time

Consolidation

Simplicity

Control

Efficiency

Unlocking AI's True Potential

Vector **Space** Scalability

Trillions of vectors, without massive memory footprint

Vector **Search** Scalability

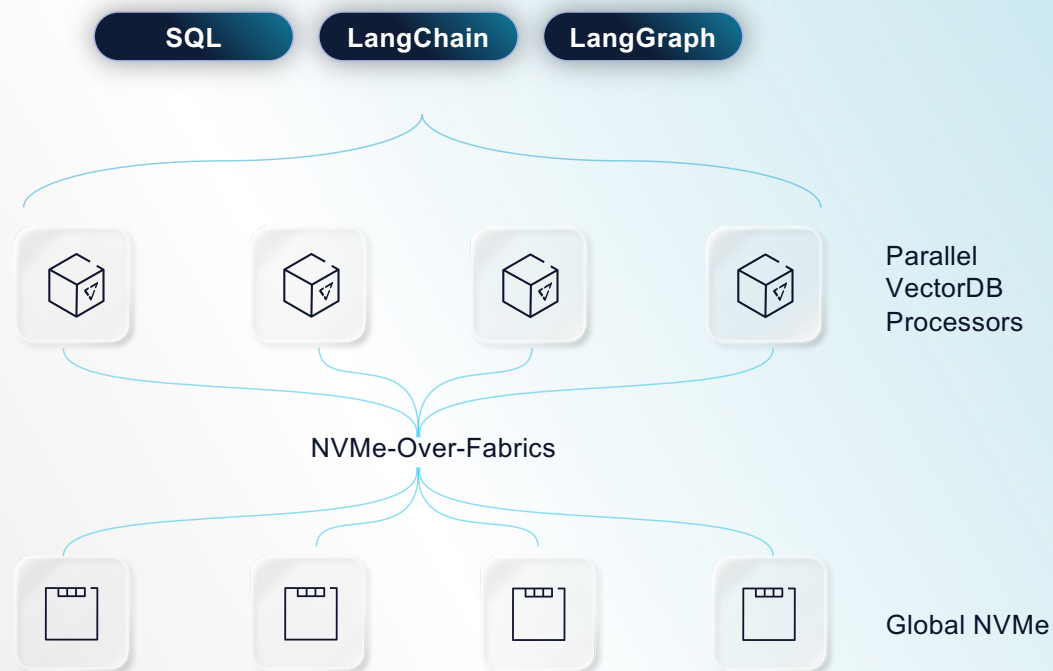
Real-time search, at scale with hierarchical clustering

Vector **Insert** Scalability

Real-time inserts, at scale with continuous background clustering

Data **Security** Atomicity

Vector ACLs match data ACLs. Adjust in real-time.



VAST Event Broker

Streaming, Analytics &
AI in One Unified Platform

VAST



Real-time analytics on events

Kafka topics are automatically stored as VAST DataBase tables for instant SQL querying



Reliable and easy to manage

Natively integrated into the VAST DataEngine—no complex clusters or external brokers



Save time and money

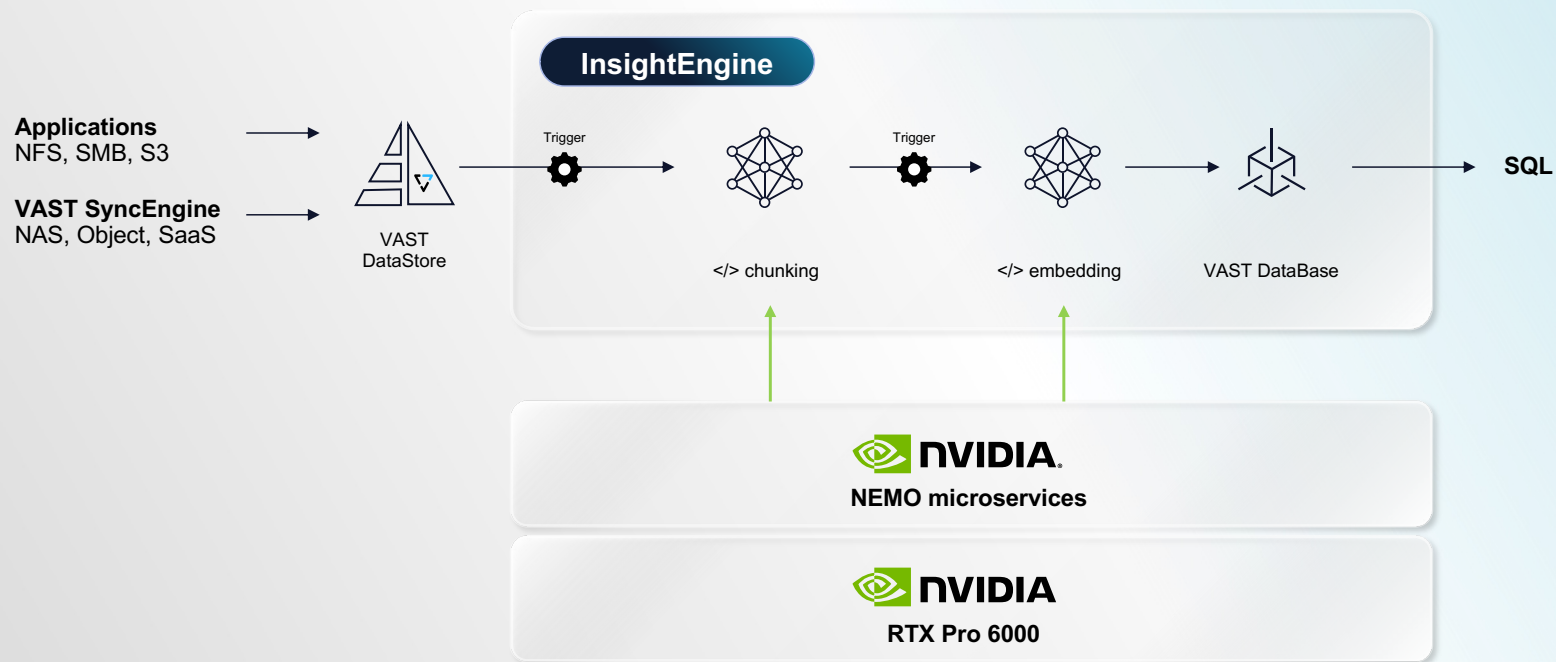
VAST simplifies infrastructure scale and complexity, while eliminating data triplication to reduce cost



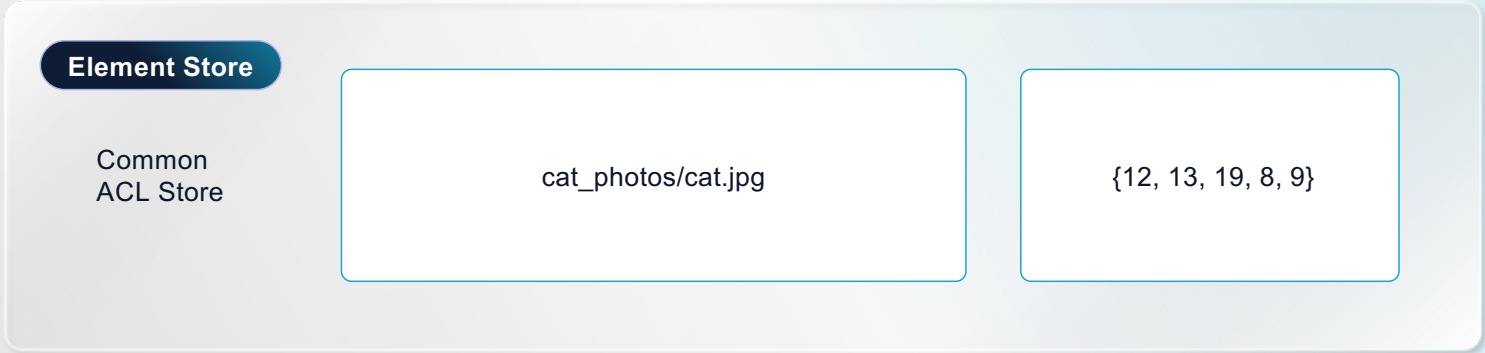
The fastest, most scalable Event Broker

Achieves up to 1M events/sec per CNode with sub-4ms latency—scales linearly without bottlenecks or complex tuning.

A Unified Engine To Ingest and Contextualize Unstructured Data

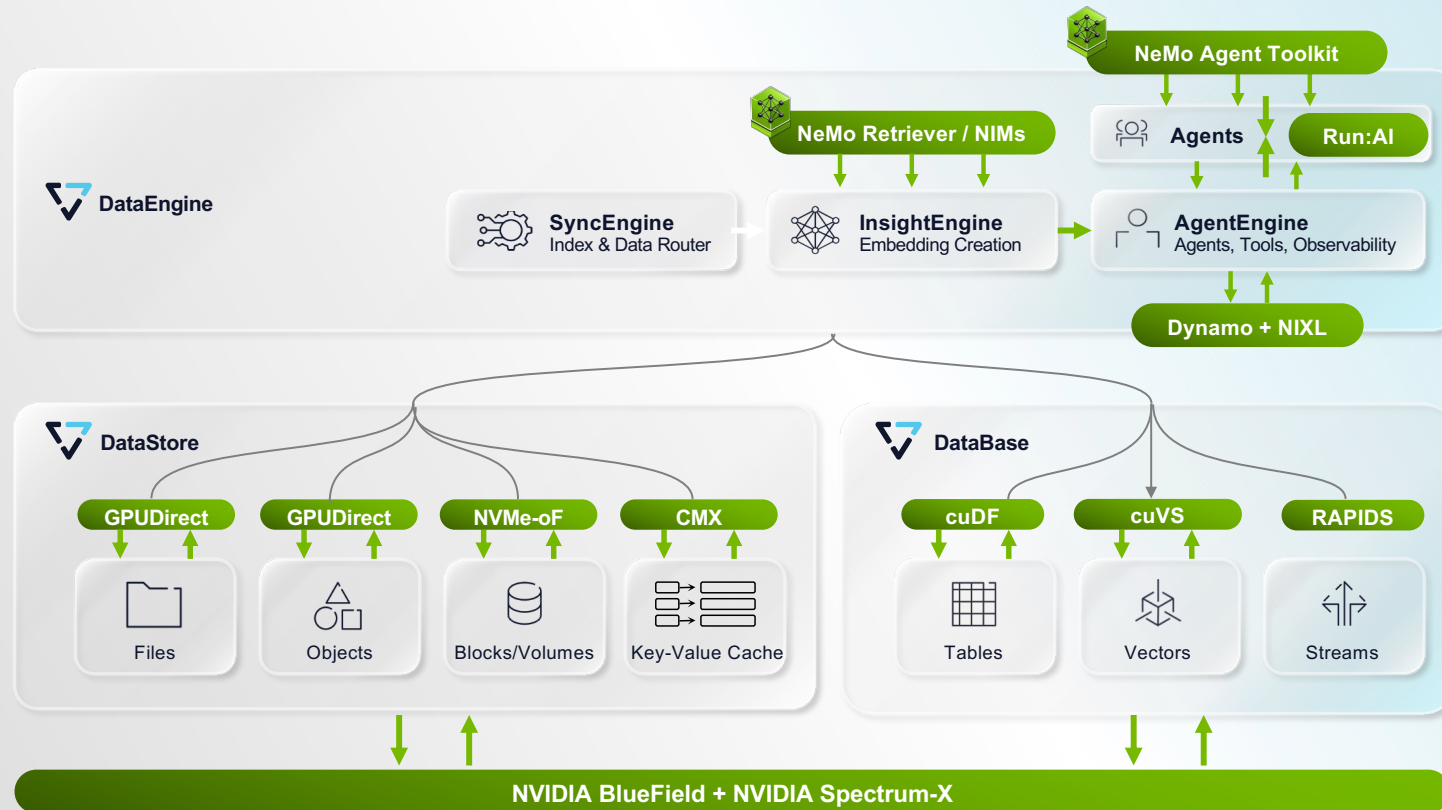


Unified Data & Vector Data Access Policies

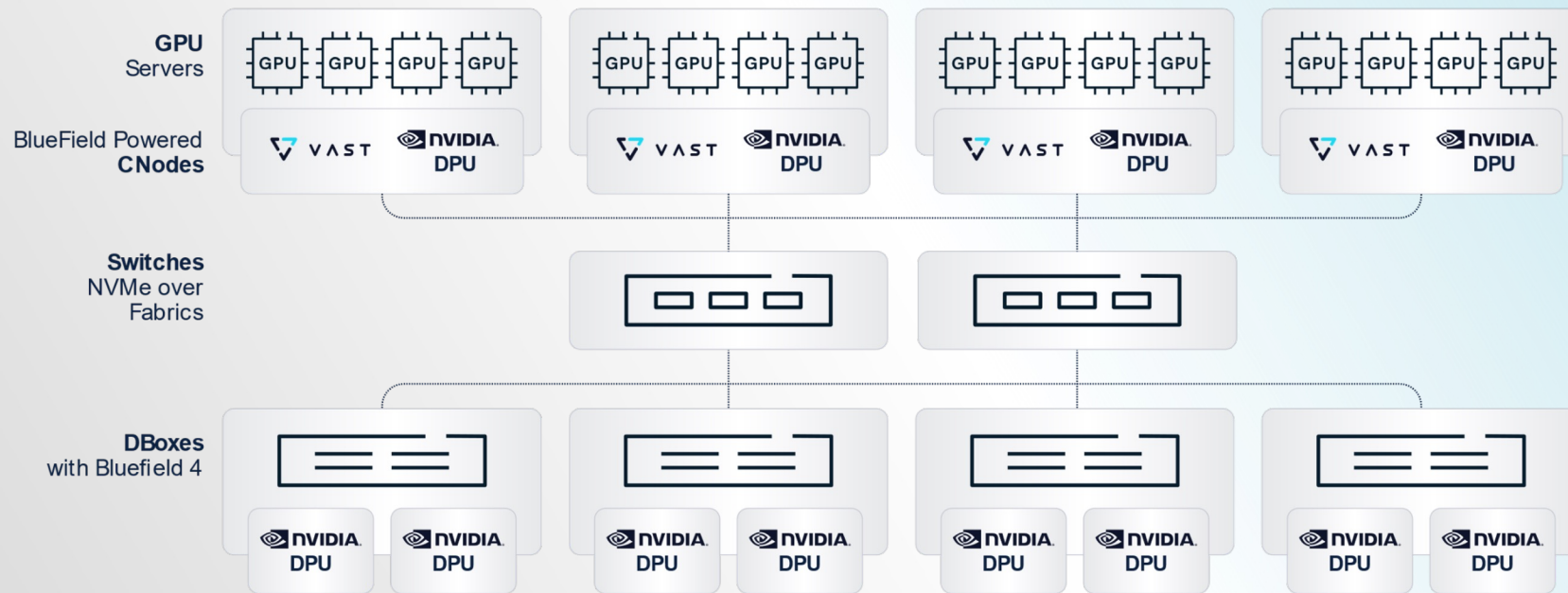


ROW_ID	vastdb_path_auth	users	groups	...	vector
1234	/path/to/original/file1	*	Marketing, Sales	...	[1,2,3,4]
1235	[/path/to/original/file2, /path/to/original/file4]	Amy, John	HR, Finance	...	[5,6,7,8]

VAST & NVIDIA: Collaborating At Every Level Of The Stack



CMX Architecture with VAST



VAST's Parallel DASE Architecture Enables Hosts To Have Dedicated VAST Servers Without Introducing Cross-Talk Across Hosts

